Stack Scout: viva notes

Live app: https://tech-stack-scout-topaz.vercel.app | Code: https://github.com/manasvinipant-cloud9/tech-stack-scout | Components map: https://tech-stack-scout-topaz.vercel.app/components-map

1. The 30-second answer

Stack Scout watches a list of target companies for a sales rep. For each one it gathers live public engineering signals (GitHub, the company website, job boards and the engineering blog), has Gemini Flash write an engineering summary and a personalised first line, scores how well the company fits the product (cloud, microservices, language), and refreshes everything weekly. Every claim carries a source, and missing evidence is shown as missing, never as a negative finding.

2. Key facts to have ready

Item	Detail
Problem	From the brief "Tech Stack Scout": a sales rep spends 2 to 3 hours researching 20 companies; reps who cite something real close about 30% more.
Stack	Next.js 15 (App Router) with React 19, TypeScript and Tailwind CSS v4; Neon serverless Postgres; Gemini Flash; Vercel hosting and Cron.
AI	Gemini Flash only (google/gemini-3.8-flash, fallback google/gemini-3.7-flash) through OpenRouter, restricted to Google's own providers. The Gemini API can be used directly instead. No Claude or other model is called by the app.
Live sources	GitHub REST API, company homepage, Greenhouse / Lever / Ashby public job boards (titles and descriptions), engineering blog RSS or Atom feeds.
Database tables	target_companies, research_runs, account_notes, app_settings.
Pages	Overview, Accounts, Account Intelligence (per company), Research & Signals, Outreach Studio, Settings, Call brief, Components map.
Scale tested	20 real companies researched on the deployed app, about 15 to 20 seconds and about 2 cents of AI cost each.
Schedule	Vercel Cron, Mondays 06:00 UTC, calling a secret-protected endpoint. Stale threshold 7 days and per-run limit 12 accounts, both editable in Settings.

3. What happens when one company is researched

- The user adds a company (name and domain, optional GitHub organisation) or the weekly job picks it up.
- The collector fetches four sources in parallel with timeouts: GitHub organisation and repositories, the homepage, job boards (reading up to 15 engineering job descriptions and counting tool mentions), and the blog feed.
- All evidence goes to Gemini Flash in one prompt, with a JSON schema for the answer: technologies (confirmed or inferred, with source), architecture style, five fit reasons, three discovery questions, a two-paragraph summary, and a first line plus the signal it is built on.
- An evidence guard downgrades any claim that cites a source we never collected.
- The fit score is calculated in code: the strength (1 to 3) of sourced fit reasons divided by 15. Unsourced reasons score 0.
- The ICP checklist (cloud, microservices, language) is evaluated in code against the targets saved in Settings.
- The result and a run record (what changed since last time) are saved in Neon, and the pages read from there.

4. Where things live in the code

File	What it does
lib/signals.ts	Collectors for GitHub, website, job boards (titles and descriptions) and blog feeds.

File	What it does
lib/llm.ts	The single place that calls a model: OpenRouter or Gemini, retries, fallback, structured JSON.
lib/gemini.ts	The analysis prompt, the JSON schema, and the evidence guard.
lib/research.ts	Runs one research job; computes the fit score; logs the run.
lib/checklist.ts	ICP checks for cloud, microservices and language.
lib/coverage.ts	Signal states: evidence found, partial, no evidence found, not researched, research failed.
lib/actions.ts, lib/insights.ts	The prioritised next-actions queue and the change feed.
lib/outreach.ts	Regenerates the opening line from one verified signal.
lib/db.ts, lib/settings.ts	All database access and the saved settings.
app/api/cron/refresh	The weekly job.
app/(app)/...	The pages. components/ holds shared UI such as the account table, drawer, command palette and charts.

5. A live demo plan (about 4 minutes)

- Open Overview: point at the hero metric, then the Recommended next actions queue. Say why each item is there.
- Click Fit distribution (Strong) to jump to a filtered Accounts list. Show the filters and the URL changing.
- Click a row to open the quick-view drawer; press j and k to step through accounts.
- Open a company (for example Vercel): read the summary, show the sources linked under it, then the fit explanation and the technology map. Click a technology to show its evidence.
- Press Regenerate on the opening line and show the new signal and source underneath. Copy it.
- Open Research & Signals: show the matrix and explain the five states, especially no evidence found versus research failed.
- Open Settings: change nothing, but explain that ICP targets and the stale threshold are saved to the database and used by the weekly job.
- Open /components-map and walk through the diagram left to right.

6. Likely viva questions, with model answers

Product and requirements

Q: Who is the user and what problem does this solve?

Nikhil, an enterprise account executive selling a developer productivity tool. His product fits only some teams, so before a first call he needs their cloud, language, architecture and engineering activity. Doing that by hand for 20 companies takes 2 to 3 hours. Stack Scout does it automatically, keeps it fresh weekly, and gives him an evidence-backed first line.

Q: How does the app meet each requirement in the brief?

Live external data: four public sources. Gemini Flash for all AI: one module only calls Gemini Flash. Summary and first line: generated from evidence, line regenerable. Relevance in cloud, microservices and language: the ICP checklist. Weekly refresh: Vercel Cron. Add and remove without code: dialogs on the Accounts page. Public URL, repository and components map: delivered.

Q: Why did you not use Supabase, since the brief mentions it?

We used Neon, which is also Postgres, because it was already connected through the Vercel integration and working. All database access is in one file (lib/db.ts), so moving to Supabase means changing the driver and the connection string. I would say this openly rather than hide it.

AI and trust

Q: Why Gemini Flash?

The brief requires it, and it suits the job: fast, inexpensive (about 2 cents per account here) and able to return structured JSON that matches a schema.

Q: How do you stop the model from hallucinating?

Four layers. The prompt allows only the supplied evidence. Each technology must be marked confirmed or inferred with a source. An evidence guard downgrades any claim citing a source we did not collect. And the fit score is computed in code from sourced reasons only, so an unsupported claim cannot raise it.

Q: What is structured output and why use it?

We give the model a JSON schema and it must answer in that shape. That makes the response predictable to parse and lets us validate fields such as the allowed source names.

Q: Why route through OpenRouter?

We started by calling the Gemini API directly. A model we chose was retired, then we hit overload and quota errors and a rejected key. Routing through OpenRouter, locked to Google models and Google's own providers, kept the app on Gemini Flash while removing those failures. The direct Gemini route still exists as an alternative.

Q: What data is sent to the model?

Only public information about a company that we collected: repository statistics, job posting text, blog post titles, homepage hints. No personal data and no private data.

Q: Could the first line be wrong or generic?

It can be imperfect, which is why it always shows the specific signal it is built on and the source, so the rep can judge it before using it. Regenerating gives another angle. It is designed to cite a concrete observation, not a template.

Scoring and logic

Q: How is the fit score calculated?

Each fit reason is rated 1 to 3 for how directly it addresses the product. The score is the sum of the sourced reasons divided by 15, as a percentage. A reason with no public source scores 0, so missing evidence never inflates it.

Q: How is the ICP checklist decided?

Cloud matches when a target cloud appears in confirmed evidence or in job descriptions. Language matches when a target language is at least 25% of public code, and is partial below that. Microservices matches when the architecture judgement is microservices and is sourced, and is partial when only inferred. Each shows match, partial, no match or no evidence.

Q: Why is there a difference between no evidence found and research failed?

No evidence found means we searched and nothing public turned up, which is not proof the company lacks it. Research failed means the request itself did not complete. Treating them the same would mislead the user.

Data and architecture

Q: Why Next.js and server components?

Pages read the database on the server and send finished HTML, which is fast and keeps keys off the browser. Small client components handle interaction such as filters, the drawer and dialogs.

Q: Why store analysis and signals as JSON in Postgres?

The shape of the evidence evolves as we add sources. JSONB lets us store it alongside the company without a migration for every new field, while the important queryable fields stay as normal columns. We fill defaults when reading older records.

Q: What does the research run history give you?

A record per refresh with the new fit score, the previous one and what changed. It powers the activity feed and the What changed insights, and will allow real trend charts once enough runs exist.

Automation and operations

Q: How does the weekly refresh work?

Vercel Cron calls /api/cron/refresh every Monday at 06:00 UTC. The endpoint requires a secret. It selects accounts that are new, stale or failed, oldest first, up to the saved limit, and researches two at a time within a time budget.

Q: What if an account has been researched recently?

It is skipped by the weekly job unless it failed. Anyone can still refresh it manually.

Q: How would this scale to 200 companies?

Raise the batch size and run more than one cron per week, or move to a queue so each account is its own job. The GitHub token already lifts the rate limit to 5,000 requests an hour. AI cost is linear at about 2 cents per account.

Security and ethics

Q: How are secrets handled?

API keys, the database URL and the cron secret are environment variables on the server only. The repository was scanned for secrets before it was made public, and database queries are parameterised.

Q: The app has no login. Is that a problem?

It is a deliberate trade-off for evaluation, because the brief asks for a working public URL. Anyone can add or refresh companies, which spends AI credit. For real use I would add authentication, such as a shared passcode or sign-in, and rate limiting.

Q: Is it ethical to collect this data?

It uses only public sources and public APIs, with timeouts, and does not log into anything or collect personal data. Every finding links back to the public page it came from.

Process, testing and limits

Q: How did you test it?

TypeScript checks and a production build on every change; 20 real companies researched on the live app; browser tests of filters, the drawer, the command palette and settings; layout checks at phone, tablet and desktop widths; and production checks that every route returns success and that the AI paths work end to end.

Q: What was the hardest problem?

Making the AI trustworthy and the deployment reliable. For trust, the evidence guard and code-computed score. For reliability, handling model retirement, overload, quota and key problems by retrying, falling back and routing through a gateway.

Q: What are the limitations?

Only public signals are visible. Job boards are found by guessing the board name from the domain. GitHub data covers the 60 most recently active repositories. Model judgements can be wrong, so each carries a source and confidence. There are no trend charts yet because run history is new.

Q: How much of this did you write yourself?

It was built iteratively with an AI coding assistant (Claude), which is worth stating plainly. The decisions were ours: the scope, choosing Neon, routing through OpenRouter, the evidence-first design, and the redesigns. Before the viva, read the files in section 4 so you can explain each one in your own words.

Q: What would you build next?

Trend charts from run history, authentication, CRM sync, conference-talk signals, and broader job-board coverage.

7. Glossary

Term	Meaning
ICP	Ideal customer profile: the kinds of company the product is relevant to. Here: target clouds, microservices, and target languages.
Structured output	Asking the model to answer in a fixed JSON shape so the result can be validated.
Evidence guard	Code that downgrades any model claim that cites a source we did not collect.
Serverless	Code that runs on demand on the host (Vercel) without a server you manage.
Cron	A scheduled job. Vercel Cron triggers the weekly refresh.
JSONB	Postgres's binary JSON column type, used for stored evidence and analysis.
ATS	Applicant tracking system, such as Greenhouse, Lever or Ashby, which hosts public job boards.

Term	Meaning
Master-detail	A list beside a detail view, so you can inspect an item without losing your place in the list.

8. Narration script, with timestamps

[0:00] 01. Stack Scout

Hello, and welcome. This video walks through Stack Scout, an AI powered tech stack intelligence system for business to business sales. It covers the problem we were given, how the system works from end to end, how artificial intelligence is used and kept honest, what the finished product looks like, how it was built step by step, the problems we hit and how we fixed them, and where its limits are. It is deployed live on Vercel, and the code is public on GitHub.

[0:29] 02. Sales reps lose hours researching how a company builds

Here is the problem. Nikhil is an enterprise account executive at a startup that sells a developer productivity tool. His product is only relevant in specific contexts: certain cloud platforms, microservices, or particular programming languages. Before a first call he researches each company by hand, reading engineering blogs, job descriptions, conference talks and public code. For a sprint of twenty companies that takes two to three hours, and the brief says he closes about thirty percent more deals when he mentions something real about how the company builds. So the task is to keep his watchlist automatically current, and to give him a personalised opening line for each company.

[1:12] 03. What the brief asked for, and where it is met

The brief translates into a clear checklist. Use live external data. Use Gemini Flash for every AI task. Show languages, cloud tooling and engineering activity for each company. Write a two paragraph summary and a first line that is not a fill in the blank template. Show relevance across cloud, microservices and language. Refresh weekly. Let the user add and remove companies without touching code. And deliver a public Vercel link, a GitHub repository and a components map. Every one of these is implemented, and the table shows where.

[1:47] 04. How the system fits together

This is the components map. On the left are four live public sources. A signal collector gathers them in parallel. The AI layer, using Gemini Flash, turns the evidence into a structured analysis. An evidence guard and scoring step then check that analysis, so nothing is claimed without a source. The result is stored in Neon Postgres and displayed by the Next J S app. Vercel Cron triggers the whole pipeline every week, and the same pipeline runs on demand whenever a user refreshes an account. The bottom lane shows how the buttons in the app call it.

[2:24] 05. Four public sources, gathered in parallel

The system uses four live public sources, and none of them need a paid key. From GitHub it finds the company's organisation and reads the language mix, how many repositories were pushed to in the last ninety days, and the star count. From the company website it reads response headers and script hosts to spot technology hints. From the public job boards of Greenhouse, Lever and Ashby it reads open engineering roles, and goes further by reading up to fifteen job descriptions to count mentions of tools such as AWS, Kubernetes, Go and microservices. And it discovers the engineering blog's feed to see recent posts and publishing cadence. One principle runs through all of this: not finding something is never treated as proof that the company does not use it.

[3:13] 06. Gemini Flash writes the analysis. Code decides what to trust.

Now the artificial intelligence. Every model call in the application goes through a single module, which guarantees only Gemini Flash models are used. There is no Claude or other model anywhere in the app. The model is asked for structured J S O N, so the output is predictable. It returns a technology inventory, a judgement on whether the company runs microservices, five reasons the product fits, three discovery questions for the sales call, the two paragraph summary, and the personalised first line. That first line can be regenerated on demand, and it always comes with the specific signal it was built on and the source of that signal. Calls are routed through OpenRouter, locked to Google's models and Google's own infrastructure, with retries and a second Gemini Flash model as fallback.

[4:02] 07. Evidence first: the model suggests, the evidence decides

This is the part we care most about, because language models can sound confident and be wrong. So the model only suggests, and the evidence decides. Every technology is marked as confirmed or inferred, with its source. An evidence guard checks the output, and if the model cites a source that was never collected, the claim is downgraded to inferred with no source. The relevance checklist for cloud, microservices and language shows match, partial, no match, or no evidence. The fit score is calculated in code from the sourced reasons only: each reason is rated one to three, the sum is divided by fifteen, and a reason without a public source scores zero, so missing evidence can never inflate a score.

[4:47] 08. Neon Postgres is the source of truth

For persistence we use Neon serverless Postgres. The target companies table holds the watchlist, the cached analysis, the signals, the fit score, the summary and the opening line. A research runs table records every refresh and what changed. Notes are stored per company, and a settings table holds the ideal customer profile targets, the stale threshold and the weekly limit, all editable from the app. One thing worth being upfront about: the brief mentions Supabase, but we used Neon. Both are Postgres, Neon was set up directly through the Vercel integration, and all database access is in one file, so moving to Supabase would mean swapping the driver and the connection string.

[5:29] 09. A weekly refresh keeps research from going stale

Automation works like this. Vercel Cron calls a protected endpoint every Monday at six A M U T C. The endpoint needs a secret, so the public cannot trigger it. It picks accounts that are new, stale or failed, oldest first, up to a saved weekly limit, and researches two at a time inside a time budget so the function never overruns. Each run is recorded with what changed. Stale means older than seven days by default, and that is editable in settings. Users can also refresh any account on demand. In our tests one account costs about two cents of AI usage.

[6:23] 10. A command centre, not a dashboard of numbers

Now the product itself. The Overview is a command centre. The hero metric is the number of strong ideal customer profile matches, with a breakdown of how many accounts match three, two, one or none of the criteria. The main area is a queue of recommended next actions. It is not just a leaderboard. It mixes accounts that are ready for outreach, accounts where fit evidence still needs completing, and accounts whose research needs refreshing. Each item shows the match indicators, a one line reason taken from the evidence, how fresh the research is, and a clear next step.

[6:57] 11. What changed, and the shape of the watchlist

Further down, a feed shows what changed: score movements detected on refresh and new engineering blog posts, all computed from stored records. Three charts answer real questions. Which clouds and languages does my watchlist run on, compared with my targets? And which high fit accounts are also hiring? Accounts with no public job board are marked unknown rather than plotted as zero, because we never present missing data as a finding.

[7:23] 12. A watchlist you can search, filter and manage

The Accounts page is the working watchlist. You can search by name or domain, switch between five saved views, filter by match, fit, status, freshness, cloud and language, and sort by several fields. Users can add a company, remove one with a confirmation step, refresh research, or run bulk actions on a selection. All of the filter state lives in the web address, so any view can be shared or bookmarked. Clicking a row opens a quick view drawer so you keep your place in the list, and the full profile remembers which list you came from, with previous and next buttons.

[7:59] 13. The company dossier: summary, opening line, evidence

Each company gets a dossier. At the top is the two paragraph engineering context written by Gemini, with the public sources it used linked underneath so anyone can verify them. On the right is the opening line. Beneath it you can see exactly which signal the line is built on and where that signal came from, and you can regenerate or copy it. There are also three discovery questions the public evidence cannot answer, and key numbers that are computed, not generated.

[8:26] 14. Why it fits, and the technology map

Lower on the same page, the fit explanation shows each of the three criteria with its evidence and a link to the source. Every fit reason can be expanded to see the evidence behind it, and missing evidence is shown explicitly rather than hidden. The technology map groups what was found into cloud, languages, and frameworks, with real logos, and clicking any technology reveals the evidence and source for it. There is a language share bar, engineering activity, and a methodology panel explaining how the research was done and how confident it is.

[9:01] 15. Research & Signals, Outreach Studio, Settings

Three supporting workspaces complete the product. Research and Signals shows coverage across the whole watchlist in a matrix with five honest states: evidence found, partial, no evidence found, not researched, and research failed. It also shows freshness, evidence gaps and a log of every refresh. Outreach Studio is a focused page where you choose a company, review the observations a line could rest on, and generate and copy an opening line. Settings lets the user edit the ideal customer profile targets, the stale threshold and the weekly limit, and these really are saved and used by the weekly job.

[9:38] 16. A deliberate design system, in dark and light

On design. The interface evolved through three versions: a simple dashboard first, then a light editorial design from a design handoff, then the premium dark system you see now, which also has a light theme. Design patterns were used only where they help. A bento layout for the overview, command centre style actions, Swiss grid style tables, a master and detail drawer for company research, and progressive disclosure so detail is one click away. Glass blur is used only for floating elements like the command palette and dialogs, never on tables or charts. Every chart answers a real question, there are no invented trends, and the app works from a phone to a desktop with accessible labels and keyboard focus.

[10:21] 17. From problem statement to production, in steps

Here is how it was built. We started by reading the brief and choosing the stack: Next J S with Tailwind, Neon for the database, Gemini for the AI, and Vercel for hosting. Then came the database, the GitHub signal collector, the first Gemini analysis and a dashboard. Next the repository, the Vercel deployment and the Neon connection. We added job boards and themes, then the weekly cron, the relevance checklist, and blog and job description signals. The interface was redesigned twice, and the AI calls were routed through OpenRouter. Later came charts, the command palette, the quick view drawer, and research on twenty real companies, followed by the components map and this walkthrough. It was built iteratively with an A I coding assistant, and every step was type checked, built, deployed and verified.

[11:10] 18. What went wrong, and what we did about it

Not everything worked first time, and these are the real problems we hit. Deployments were blocked because the commit email was not linked to GitHub, so we used the GitHub no reply address. The Gemini model we first chose was retired, so we updated it and added a fallback. Overload and quota errors led to retries, a fallback model and plain English error messages. When an A P I key was rejected, we routed the calls through OpenRouter, restricted to Google's models. Older records crashed pages after the data model grew, so defaults are now filled in on read. A client bundle accidentally included the database driver, a mobile layout overflowed, and a font feature distorted punctuation. We fixed each of these. And Vercel occasionally did not start a deployment, which a fresh push resolved.

[12:01] 19. How we know it works

How do we know it works? Every change was type checked and built before it was pushed. Twenty real companies were researched live on the deployed app, taking fifteen to twenty seconds each. We tested interactions in a real browser: that filter counts match the overview, that the quick view steps through the list, that the command palette and sidebar behave, and that settings save and read back. Layouts were checked at phone, tablet and desktop widths with no horizontal overflow. In production every route returns success, and the A I paths for research, regenerating a line and saving settings were verified end to end.

[12:38] 20. What it cannot honestly claim

It is important to be honest about the limits. The system only sees public signals, so private repositories and internal tooling are invisible. The phrase no evidence found does not mean absent, and the interface says so. Job boards are discovered by guessing the board name from the domain, so some companies show no hiring data. GitHub data covers the sixty most recently active repositories. Model judgements can be wrong, which is exactly why every claim carries a source and a confidence level. There are no trend charts yet because run history only started recently, and the app has no login because it is public for evaluation.

[13:18] 21. Short answers to prepare

Finally, some questions you may be asked, with short answers. Why Gemini Flash? It is fast, costs about two cents per account, and supports structured output. How do you stop hallucination? Claims need a collected source, an evidence guard downgrades the rest, and the score counts only sourced reasons. Why Neon and not Supabase? Both are Postgres, Neon was already connected through Vercel, and swapping needs only the driver and connection string. How does the weekly refresh work? Vercel Cron calls a secret protected endpoint on Mondays and refreshes new, stale or failed accounts. And what would you build next? Trend charts from run history, a login, C R M sync, conference talk signals and wider job board coverage.

[14:06] 22. Stack Scout

That is Stack Scout, from the problem statement to a deployed product. The live app, the public repository and the components map are all linked on screen. Thank you for watching, and I am happy to take any questions.